

SinSiege

Developer Handover & Codebase Guide

Current state of the code, the project architecture, and how to use the custom Unity tooling (Sheet Fetcher, Enemy Creator, Wave Editor).

Unity 6000.3.8f1 · Universal Render Pipeline 2D · FMOD Audio
Document generated June 2026

Contents

1. Introduction	4
1.1 Tech stack	4
1.2 Opening & running the project	4
1.3 Source control	4
2. Project Layout	4
2.1 Runtime script groups	5
3. Architecture & Current State	5
3.1 Bootstrapping & lifecycle	5
3.2 Scene managers at a glance	6
3.3 Wave / Route / Spawning system	6
Data model	6
Runtime flow (WaveManager)	7
3.4 Enemies	7
3.5 Towers, combat & economy	8
3.6 Shared registries	8
3.7 Localization	8
3.8 Known rough edges	8
4. Custom Tools	8
4.0 Menu map	9
4.1 Sheet Fetcher	9
Opening & usage	9
Column mappings	9
Quick recipe	10
4.2 Localization Sync	10
4.3 Enemy Creator	11
Fields	11
What "Create Enemy" does	11
4.4 Wave Editor	11
Waves mode	12
Segments mode	12
Routes mode	12
4.5 Supporting editor utilities	13

5. Common Workflows	13
5.1 Add a new enemy	13
5.2 Create or edit a wave	13
5.3 Build a new level	13
5.4 Tune balance, characters or upgrades	13
5.5 Add a tower	14
5.6 Add or translate UI text	14
6. Conventions, Gotchas & Cheat-Sheet	14
6.1 Conventions to respect	14
6.2 Easy mistakes	14
6.3 Tools cheat-sheet	14
6.4 Key file index	15

1. Introduction

SinSiege is a 2D tower-defense / wave-survival hybrid built in Unity 6. It carries a seven-deadly-sins theme (the first playable region is “Lust”).

The player controls a hero character on a map and also **builds towers** on fixed zones. Together they defend a central **Base** against **waves** of enemies. Enemies travel along authored **routes** toward the Base, while a parallel **horde** mechanic spawns enemies from the screen edges that chase the player directly. Killing enemies drops **Souls** (the in-game currency) used to build/merge towers and buy upgrades. Characters, towers, upgrades and all UI text are data-driven and localized into 12 languages.

1.1 Tech stack

Area	Choice	Notes
Engine	Unity 6000.3.8f1	Open with this exact version via Unity Hub.
Rendering	URP 17.3 (2D Renderer)	2D feature set 2.0.2, Pixel Perfect, Sprite tools.
Camera	Cinemachine 3.1.3	Player cam + build-mode cam, swapped at runtime.
Input	Input System 1.18	InputSystem_Actions.inputactions at Assets root.
UI / Text	uGUI 2.0 + TextMeshPro	Localized via LocalizedText components.
Audio	FMOD Studio	External integration; see Assets/Plugins/FMOD and Assets/Fmod.
Post / Capture	Post Processing 3.5, Recorder 5.1	Recorder used for trailer/demo capture.

1.2 Opening & running the project

1. Install **Unity 6000.3.8f1** (Unity Hub → Installs). The FMOD and URP packages restore automatically from Packages/manifest.json.
2. Open the repository root folder as a project. The first import is slow (it regenerates Library/).
3. Press Play from **Assets/Scenes/MainMenu.unity** for the full flow, or open **Assets/Scenes/Lust1.unity** to jump straight into gameplay.

i Scenes in the project

MainMenu — title screen, character/level select. **Lust1** — the main gameplay level. **Gokhan Testing** — a scratch/experimental scene, not shipped. Use the **Scenes** menu (Ctrl+G) to jump between them quickly.

1.3 Source control

- Standard Unity .gitignore — Library/, obj/, Logs/, Builds/ and the generated *.csproj / *.sln files are not committed.
- .gitattributes only normalizes line endings and pins FMOD bundle/plist files to LF. **Git LFS is not currently configured** — large art/audio binaries are stored directly.
- Asset .meta files **must** be committed alongside their assets.

2. Project Layout

Everything game-specific lives under Assets/. The folders you will touch most often:

Path	What lives here
Assets/Scripts	All runtime gameplay code (managers, mechanics, data, UI, towers, enemies).
Assets/Scripts/Editor	Editor-only code that ships next to gameplay: Sheet Fetcher, Localization Sync, WaveSO inspector.
Assets/Editor	Standalone editor tools: Enemy Creator, Wave Editor, Scene Quick Access, button drawer.
Assets/Prefabs	Enemies, Towers, Bullets, Coins, Levels, Player, UI, Persistent, Props.
Assets/Resources	Assets loaded by name at runtime: ReferencesSO, LocalizationData, Upgrades/, Characters/, Materials/.
Assets/Scriptable Objects	Authoring data assets: Waves/ (per-level WaveSO files) and Routes/.
Assets/Scenes	MainMenu, Lust1, Gokhan Testing.
Assets/Sprites / Animation / Materials / Shaders	Art, animation clips/controllers, materials and shaders.
Assets/Fmod / SinSiegeFMOD / FMODAssets	FMOD banks, project link and generated audio GUIDs.

⚠ Two editor-script locations

Editor tooling is split between **Assets/Editor** and **Assets/Scripts/Editor**. Both compile into Unity's editor assembly purely because each path contains a folder named `Editor`. When adding a new tool, drop it in either — but be consistent and never reference editor code from runtime scripts (it will break player builds).

2.1 Runtime script groups

Folder under Scripts/	Responsibility
System	Bootstrapping & cross-cutting: LevelInitializer, PersistentManager refs, GameState, MasterDictionary, ReferencesSO/Refs, Config.
Managers	Per-scene services: GameManager, WaveManager, BuildManager, TowerManager, UpgradeManager, Music/Sound, Save/Scene/Persistent.
Wave Related	WaveSO + WaveSpawnData, WaveGroup + WaveSlot (the data the Wave Editor authors).
Data	LevelData (routes/segments/waves on the level prefab), CharacterData.
Enemies / Bosses	Enemy.cs movement & combat; Boss, MammonArm.
Towers	Base, Tower, TowerGeneric, TowerChapel, TowerManager, TowerZone.
Mechanics	Player, Shop, Coin, Upgrade(Data), Minimap, Combat/ (Bullet, FireMethods).
UI	Combat HUD, build/merge menus, menu buttons, cards, sliders.
Sheets Related	Localization runtime: LocalizationManager/Data, LocalizedText, LanguageDropdown.

3. Architecture & Current State

This section describes how the running game is wired together — read it before changing managers or the spawning pipeline.

3.1 Bootstrapping & lifecycle

The game uses an explicit **Init()** pattern rather than relying on Unity's Awake/Start ordering. Two objects drive startup:

- **PersistentManager** — a DontDestroyOnLoad singleton (in the Persistent prefab) that survives scene loads. It owns **GameState**, **LocalizationManager**, **SaveManager** and **SceneManager**. Each of those exposes a static Instance => PersistentManager.Instance.X accessor.
- **LevelInitializer** — a per-gameplay-scene singleton. On Awake it finds every scene manager (FindManagersAndObjects()), calls each manager's Init() in a deliberate order, reads the chosen level index from GameState, and loads it.

```
// LevelInitializer.Awake -> FindManagersAndObjects() calls, in order:
Player.Init();
ArrowManager.Init(); BuildManager.Init(); GameManager.Init();
MouseManager.Init(); WaveManager.Init();

// Then SetLevelIndex() reads PersistentManager.GameState.LevelIndex,
// and LoadLevel() pushes the matching LevelData into the WaveManager:
WaveManager.SetLevelData(levelDatas[levelIndex]);
WaveManager.ResetWavesAndRoutes();
WaveManager.GetWavesAndRoutesFromLevelData(); // caches route paths
```

i How a level is selected

From the menu, SceneManager.StartLevelWithData(sceneName, levelIndex) stores the index on **GameState** (firing OnLevelChanged) and loads the scene. **LevelInitializer.levelDatas** is an ordered list assigned in the Inspector; the level index selects which LevelData the WaveManager uses. Keep that list, the scene Build Settings and the menu's indices in sync.

3.2 Scene managers at a glance

Manager	Role
GameManager	Central gameplay hub: base health UI, Souls/coin spawning & denomination, build/combat camera swap, mouse world position.
WaveManager	Drives spawning from LevelData (see 3.3). Owns wave timer, route blocking, horde spawning, win condition.
BuildManager	Tower placement / build-mode UI (builder, manager and merge menus).
TowerManager	Live tower registry & tower-side operations.
UpgradeManager	Periodic upgrade choices (every 5th wave); applies UpgradeData effects.
MusicManager	FMOD music; SetCombatToTrue/False toggles the combat parameter.
SoundManager	FMOD one-shot SFX (e.g. enemy death).
SortingManager	2D sprite sort-order helpers.
ArrowManager / MouseManager	Off-screen target arrows; custom cursor & click routing.
SaveManager	SaveGame / LoadGame / DeleteSave (PlayerPrefs/JSON persistence).
SceneManager	Wraps Unity scene loading; StartGame, StartLevelWithData, current scene name.

3.3 Wave / Route / Spawning system

This is the heart of the game and the data the **Wave Editor** authors. The data lives on the **level prefab root** in a LevelData component.

Data model

- **LevelData** holds three authored lists: mapRoutes, waveGroups, and availableSegments (a reusable segment pool), plus spawnPoints.

- **PathSegment** = a name + a spawn point + an ordered list of waypoint Transforms (children of a `Waypoints` object on the prefab).
- **MapRoute** = a name + ordered `pathSegments`. At runtime `GetCalculatedPath()` flattens the spawn point + every waypoint and **auto-appends the Base position** (it calls `FindFirstObjectByType<Base>()`).
- **WaveGroup** = a list of **WaveSlot**. A **WaveSlot** pairs a `WaveSO` with a `routeIndex` (an index into `mapRoutes`).
- **WaveSO** (`ScriptableObject`) = the actual enemy script: pre-wave cooldown, default spawn interval, an `enemySpawns` list, optional horde config, and cached Gold/Exp totals.
- **WaveSpawnData** = one spawn entry: enemy prefab, count, optional interval override, and a stat-modification mode (`None` / `Multiplier` / `CustomValue`) with the corresponding fields.

Runtime flow (WaveManager)

1. On Init, each route's path is flattened once into `_cachedPaths`.
2. A countdown timer runs; the **Start Wave** button can skip it. When it hits zero the current `WaveGroup` starts.
3. Each non-empty slot becomes a **WaveSpawnerState**. `ExpandByCount()` flattens each entry so it appears *count* times.
4. All slots in a group run **simultaneously**, but slots sharing the same `routeIndex` are **serialized left-to-right** — an earlier same-route slot blocks later ones until it finishes (route blocking).
5. Each spawn instantiates the enemy at the route's start, assigns it the cached path and the `WaveManager`, then `ApplyConfigToEnemy()` applies multipliers or custom values.
6. Hordes (if any slot enables one) spawn from random screen edges with `followPlayer = true` while normal enemies are still alive.
7. The group is complete when all spawners are empty and no enemies remain; the wave index advances. Every 5th wave triggers `UpgradeManager.TimeToUpgrade()`. When the last group ends, `allWavesCompleted` is set.

i Route index is positional

A `WaveSlot.routeIndex` is simply the position of the route in `LevelData.mapRoutes`. Reordering or deleting routes shifts those indices — re-check wave/route assignments in the Wave Editor afterwards.

3.4 Enemies

`Enemy.cs` carries serialized base stats (`_moveSpeed`, `_health`, `damage`, `exp`, `coinValue`, `sliderOffset`) exposed read-only as `BaseHealth/BaseSpeed/...`. It moves along its assigned path (or follows the player when `followPlayer` is set), flips its sprite by travel direction, sorts by Y, spawns a world-space health bar on first damage, and on death calls `GameManager.SpawnCoins()`. `InitializeStats()` lets the `WaveManager` override stats per spawn.

- Enemy prefabs live in `Assets/Prefabs/Enemies/<Category>/` where category is a sin name (Lust, Greed, ...), plus `Generic`, `GenericHorde`, `LustHorde`, `Bosses`.
- **EnemyDatabase** (a `ScriptableObject`) lists every enemy for the inspector pickers. It has an **Auto-Find All Enemies** context-menu item that rescans the project.
- New enemies are created with the **Enemy Creator** tool (Section 4.3) as prefab variants.

⚠ Open TODO: enemies do not yet damage the Base

When an enemy reaches the end of its path, `Enemy.Update()` currently just destroys it (there is a `// TODO: Deal damage to base?`). Base health UI exists but nothing decrements it from path completion yet — this is the most visible gameplay gap to wire up.

3.5 Towers, combat & economy

- **Tower hierarchy:** Base (the HQ), Tower / TowerGeneric (stats + firing), specialised TowerChapel, plus TowerManager and TowerZone (placement zones).
- **TowerGeneric** exposes the balance fields the Sheet Fetcher writes (`towerName`, `tier`, `animationInit`, `attackRangeDefault`, `attackIntervalDefault`, `attackDamageDefault`, `bulletSpeed`, `bulletCount`, `spreadAngle`, `bulletHealth`, `bulletIsSpinning`, `isAOEBullet`, `targetTagDefault`).
- **Combat:** FireMethods (targeting, TargetTag enum), Bullet, CrossBulletImpact.
- **Economy:** currency is “Souls”. `GameManager.SpawnCoins(value, pos)` breaks a value into coin denominations (parallel `CoinPrefabs` / `CoinValues` lists).
- **Upgrades:** UpgradeData assets (UpgradeType enum: AttackInterval, Damage, BulletSpeed, BulletCount, BulletHealth, MoveSpeed, MaxHealth, Custom) drive UpgradeManager.

3.6 Shared registries

- **ReferencesSO** (in Resources, fetched via `Refs.R`) is the central registry: the character list, a color palette, and `TowerReferences[]` (name + prefab, indexed by tower ID).
- **MasterDictionary** is a static catalog of enums: GameLanguage (12), Characters (8), Towers (18), UpgradeRarity (7) and rarity colors, plus the MainMenu scene name constant.

3.7 Localization

Text is data-driven. `LocalizationData` (a ScriptableObject in Resources) holds language codes + key/value rows. `LocalizationManager` builds a fast key→string map for the current language and raises `OnLanguageChanged`; `LocalizedText` components subscribe and refresh. The data is baked from a Google Sheet with the **Localization Sync** tool (Section 4.2).

3.8 Known rough edges

⚠ Things to be aware of when taking over

- Enemies reaching the Base are destroyed without dealing damage (TODO above).
- `MasterDictionary.RarityColors` uses new `Color(0-255...)` instead of `Color32`, so those struct values are out of the 0–1 range (the parallel `Color32[]` array is the correct one).
- Two enemy-picker UIs coexist: the Wave Editor's built-in grid (scans the Enemies folder) and `EnemySelectorPopup` (uses `EnemyDatabase`). They can drift apart.
- `Level_..._OLD.prefab` and the “Gokhan Testing” scene are scratch artifacts.
- Heavy use of `FindFirstObjectByType` in Init paths — fine today, but watch it if scenes grow.

4. Custom Tools

*The project ships several editor tools that make content authoring fast. They live under the **Tools** and **Scenes** menus. This is the part most relevant to day-to-day content work.*

4.0 Menu map

Menu item	Tool	Section
Tools › Sheet Fetcher › Open Menu	Sheet Fetcher window	4.1
Tools › Sheet Fetcher › Sync Upgrades / Characters / Towers	One-click sync (uses saved URLs)	4.1
Tools › Localization › Open Downloader	Localization Sync	4.2
Tools › Enemy Creator	Enemy Creator	4.3
Tools › Wave Editor	Wave / Segment / Route editor	4.4
Scenes › Open Scene List (Ctrl+G)	Scene Quick Access	4.5
(WaveSO inspector) Calculate Total Gold & Exp	WaveSO editor button	4.5
(EnemyDatabase context menu) Auto-Find All Enemies	Rebuild enemy list	4.5

4.1 Sheet Fetcher

File: Assets/Scripts/Editor/SheetFetcher.cs

Syncs game-balance data from **Google Sheets** directly into ScriptableObjects and tower prefabs, so designers can tune numbers in a spreadsheet and pull them into Unity with one click.

Opening & usage

1. Open **Tools › Sheet Fetcher › Open Menu**. The window has three sections: **Upgrades**, **Characters**, **Towers**.
2. Paste the normal Google Sheets share link into a section's **CSV Link** field. The tool auto-converts a /edit URL into a /export?format=csv URL, so a plain share link works. URLs are remembered per section in EditorPrefs.
3. Click **Sync** to write assets, or **Debug (Print Rows)** to dump the first few parsed rows to the Console — use Debug first to confirm the columns line up.

i Sheet must be readable & column order matters

Share the sheet as **Anyone with the link – Viewer** (or publish to web). The first row is treated as a header and skipped. The separator (comma or semicolon) is auto-detected, and decimals parse under both invariant and Turkish cultures.

Column mappings

Upgrades → one UpgradeData asset per row in Assets/Resources/Upgrades:

Col	Field	Col	Field
0	upgradeID	5	value (float)
1	upgradeName	6	isMultiplier (TRUE/1)
3	upgradeLevel	8	secondaryValue
4	upgradeType (enum)	9	ternaryValue

Characters → one CharacterData asset per row in Assets/Resources/Characters (needs ≥ 7 columns):

Col	Field	Col	Field
0	id (Characters enum name)	4	damage
1	characterName	5	attackSpeed
2	fullName	6	movementSpeed
3	desc		

Towers → writes onto the TowerGeneric component of the prefab at `ReferencesSO.TowerReferences[ID]`. No new assets are created and nothing is deleted.

Col	Field	Col	Field
0	towerName	8	attackDamageDefault
1	ID (index into TowerReferences)	9	bulletSpeed
2	animationInit	10	bulletCount
3	tier	11	spreadAngle
4	mergeContent (skipped)	12	bulletHealth
5	targetTag (enum)	13	bulletIsSpinning
6	attackRangeDefault	14	isAOEBullet
7	attackIntervalDefault		

⚠ Upgrades & Characters sync is destructive

After writing, the Upgrades and Characters sync **deletes any .asset in the target folder that is not present in the sheet**. Keep those folders for synced data only — do not hand-place unrelated assets there. The Towers sync does **not** delete (it edits existing prefabs in place), but the tower **ID column must be a valid index** into `ReferencesSO.TowerReferences` or the row is skipped.

Quick recipe

1. Edit numbers in the Google Sheet.
2. In Unity: **Tools > Sheet Fetcher**, click **Debug** on the relevant section, confirm the Console rows look right.
3. Click **Sync**. Check the green “Sync Complete” log and the updated assets.
4. For towers, make sure the prefab IDs/order in `ReferencesSO` match the sheet's ID column first.

4.2 Localization Sync

File: `Assets/Scripts/Editor/LocalizationSync.cs`

A sibling of the Sheet Fetcher dedicated to UI text. Open **Tools > Localization > Open Downloader**, paste the CSV link, and click **Fetch & Bake Data**.

- It downloads the sheet and writes `Assets/Resources/LocalizationData.asset` (creating it if missing).
- Sheet shape: the **header row** is `key, en, tr, fr, ...` — the first column is the lookup key, each following column is a language code. Every bake **clears and rebuilds** all entries.
- Language codes map to `MasterDictionary.GameLanguage` in `LocalizationManager.GetLanguageCode()` (`en, tr, fr, it, de, pt, ru, pl, kr, jp, zh, tc`).
- At runtime, call `LocalizationManager.Instance.GetLocalizedValue(key)`; missing keys return `MISSING: key` so they are easy to spot.

4.3 Enemy Creator

File: Assets/Editor/Enemy Creator/EnemyCreatorWindow.cs

Creates a new enemy as a **Prefab Variant** of an existing base enemy prefab. You enter the Enemy.cs stats, pick a sprite, and either generate a looping animation from a sliced sprite sheet or assign an existing Animator Controller. Open via **Tools > Enemy Creator**.

Fields

Field	Meaning
Base Enemy Prefab	Template to make a Variant of. Must have an Enemy component on its root. Selecting one auto-fills stats and folders.
New Enemy Name	File name of the created prefab (sanitised).
Prefab Folder	Output folder (defaults to the base prefab's folder). Use ... to browse; must be inside Assets.
Enemy Stats	Move Speed, Health, Damage, Exp, Coin Value, Health-Bar Offset. "Reset stats from base" re-reads the base.
Animation mode	Create From Sheet · Use Existing Controller · None.
Sprite Sheet (sliced)	A texture imported as Sprite Mode = Multiple and sliced into frames (Create-From-Sheet mode).
Samples (fps)	Animation sample rate / frames per second for the generated clip (default 8).
Animation Folder	Where the generated .anim / .controller are written.
Animator Controller	An existing controller (Use-Existing mode).
Default Sprite	Resting sprite for the SpriteRenderer; empty = the sheet's first frame.

What "Create Enemy" does

1. If Create-From-Sheet: loads the sheet's sub-sprites (natural-sorted frame_0, frame_1, ...), builds a **looping** sprite clip at the chosen fps, and creates an AnimatorController from it.
2. Instantiates the base prefab, writes the stat overrides + default sprite + controller through SerializedObject so they are recorded as variant overrides.
3. Saves as a Prefab **Variant** at [Folder]/[Name].prefab (clip: [Name]_Move.anim, controller: [Name]_Animc.controller) and pings it in the Project window.

✓ After creating an enemy

Place it in the correct sub-folder under Assets/Prefabs/Enemies/ (the folder name drives category grouping in the picker), then run **Auto-Find All Enemies** on the EnemyDatabase so it appears in the database-backed picker. Because it is a Variant, later edits to the base prefab propagate automatically.

4.4 Wave Editor

Files: Assets/Editor/Wave Editor/ — WaveEditorWindow.cs, EnemySelectorPopup.cs, WaveSpawnDataDrawer.cs.

The main level-authoring tool: it edits a level's **waves, path segments and routes**. Open via **Tools > Wave Editor**, then pick a level from the dropdown (any prefab with a LevelData component under Assets/Prefabs/Levels).

! It edits the level live in Prefab Mode

Selecting a level opens it in Unity's **Prefab Mode** and binds to the live `LevelData`. Every edit goes through one `SerializedObject` (or an Undo-recorded Transform move), so references can't desync. **Saving** uses the normal Prefab Mode save — Auto Save, or Ctrl+S. The top bar shows a green "Saved" / amber "Unsaved" indicator.

A top toolbar switches between three modes: **Waves**, **Segments**, **Routes**. The logical authoring order is Segments → Routes → Waves.

Waves mode

- **Left timeline:** ordered **Wave** groups, each containing **spawner slots**. Use + Add Wave, + Add spawner, ↑/↓ to reorder, × to delete.
- Each slot has a colour-coded **route dropdown** and a **wave** reference.
- **Assign a wave** three ways: select a slot then click **Assign** in the Library tab; **drag a WaveSO** from the Project window onto the slot; or **Create New Wave & Assign**.
- **Right panel — Wave Editor tab:** rename the wave, see live Gold/Exp totals, set Timing (Pre-Wave Cooldown, Default Spawn Interval), edit the **Enemy Spawns** list, and toggle a **Horde** (interval + its own spawn list).
- Each spawn entry has a clickable enemy sprite (opens a grid picker of all prefabs with an Enemy under Assets/Prefabs/Enemies), a **count**, an optional per-entry **interval override**, and a **Stat Mode**: None, Multiplier (HP/Speed/Damage/Gold/Exp ×), or CustomValue (absolute Health/Speed/Damage/Gold/Exp).
- **Library tab:** all WaveSO assets grouped by folder (the current level's folder is highlighted), with search, create, delete and an assigned-count badge. New waves are created under Assets/Scriptable Objects/Waves/[LevelName].

Segments mode

- Manages the level's reusable **segment pool** (`availableSegments`).
- Create a segment, give it a name, choose a **Spawn Point** (dropdown populated from the level's Spawn Points child object), then build its ordered **waypoint list** by adding from **Available in Prefab** (children of the Waypoints child object). Reorder with Up/Dn.
- **Scene handles:** with a segment selected, click a white dot to add a waypoint, a green dot to remove it, and drag the position handles to move waypoints. Spawn points are shown in orange with a dotted line to the first waypoint.

Routes mode

- Create a **route**, name it, then build its **Segment Sequence** with + **Add Segment from Pool** (a menu of pooled segments). Reorder / delete sequence entries.
- The first segment in a route **must have a Spawn Point** (the editor warns if not).
- **Scene handles** draw each route's path in a distinct colour; the selected route's waypoints are movable.
- A route's position in the list **is** its index — that's the `routeIndex` a wave slot points at (and what `WaveManager._cachedPaths` uses).

⚠ Level prefab requirements

For Segments/Routes to work, the level prefab root needs child objects named exactly `Waypoints` (holding waypoint Transforms) and `Spawn Points` (holding spawn Transforms), and the gameplay scene needs a `Base` object (routes auto-append its position as the path end). The level's `LevelData` must also be registered in `LevelInitializer.levelDatas` to be playable.

4.5 Supporting editor utilities

- **WaveSpawnDataDrawer + EnemySelectorPopup** — a custom property drawer used when editing a WaveSO in the *normal* Inspector. It shows a large sprite button that opens a popup of enemies **categorised by sin folder**, backed by the EnemyDatabase (previews are cached). If no EnemyDatabase exists it logs an error — create one via **Create › TowerDefense › EnemyDatabase**.
- **WaveSOEditor** — adds a **Calculate Total Gold & Exp** button to the WaveSO inspector (the Wave Editor recalculates these automatically; this is for manual edits).
- **SceneQuickAccess** — **Scenes › Open Scene List** (Ctrl+G): a popup list of every scene in Assets/Scenes for fast switching (prompts to save first).
- **EnemyDatabase.FindAllEnemies** — right-click the EnemyDatabase asset → **Auto-Find All Enemies** to rescan the project and rebuild its list.

5. Common Workflows

Step-by-step recipes for the most frequent content tasks.

5.1 Add a new enemy

1. **Tools › Enemy Creator**. Assign a Base Enemy Prefab, set stats, pick a sliced sprite sheet (or a controller), click **Create Enemy**.
2. Move/keep the prefab in the right Assets/Prefabs/Enemies/<Sin>/ folder.
3. Select the EnemyDatabase asset → **Auto-Find All Enemies**.
4. The enemy now appears in the Wave Editor's spawn picker.

5.2 Create or edit a wave

1. **Tools › Wave Editor**, pick the level.
2. Open the **Library** tab, **Create New Wave** (or select one).
3. In the **Wave Editor** tab add spawn entries, choose enemies, counts, intervals and stat mode; optionally enable a Horde.
4. Add a Wave group / spawner in the timeline and **Assign** the wave to a slot; set its route.
5. Ctrl+S (or rely on Auto Save) to save the level prefab.

5.3 Build a new level

1. Duplicate an existing level prefab in Assets/Prefabs/Levels (it already has a LevelData, Waypoints and Spawn Points structure to copy).
2. Lay out waypoints and spawn points as children of those objects. Ensure the gameplay scene has a Base.
3. In the Wave Editor: build **Segments**, assemble them into **Routes**, then author **Waves** and assign them to routes.
4. Add the new LevelData to **LevelInitializer.levelDatas** (note its index), add the scene to Build Settings, and make the menu pass the matching level index via `SceneManager.StartLevelWithData`.

5.4 Tune balance, characters or upgrades

1. Edit the Google Sheet(s).

2. **Tools > Sheet Fetcher** → Debug to verify → Sync the relevant section (Upgrades / Characters / Towers). Remember the destructive delete for Upgrades/Characters.

5.5 Add a tower

1. Create the tower prefab with a `TowerGeneric` (or subclass) component.
2. Add it to `ReferencesSO.TowerReferences` at the intended ID (index).
3. Add an entry to `MasterDictionary.Towers` if it needs an enum.
4. Put its stats in the towers sheet (ID column = that index) and run the Towers sync.

5.6 Add or translate UI text

1. Add a key row (and translations) in the localization sheet.
2. **Tools > Localization > Open Downloader** → **Fetch & Bake Data**.
3. Reference the key from a `LocalizedText` component or `GetLocalizedValue(key)`.

6. Conventions, Gotchas & Cheat-Sheet

6.1 Conventions to respect

- **Init() pattern:** managers are wired by `LevelInitializer`, not by ad-hoc `Awake` order. New managers should expose an `Init()` and be added to that flow.
- **Resources by name:** `ReferencesSO` and `LocalizationData` are loaded by exact name from `Resources` — don't rename or move them.
- **Enemy folder names = categories** in pickers; keep the sin sub-folders.
- **Route index is positional** — reordering routes rebinds wave slots.
- **Editor code stays in Editor folders** and is fenced with `#if UNITY_EDITOR` where it sits beside runtime code.

6.2 Easy mistakes

- Running Upgrades/Characters sync with stray assets in the target folder — they get deleted.
- Forgetting to save the level prefab after Wave Editor changes (watch the Saved/Unsaved badge).
- Adding a level prefab but not registering its `LevelData` in `LevelInitializer` (it won't load).
- Tower sheet IDs that don't match `TowerReferences` indices (rows silently skipped).
- Creating an enemy but not re-running `EnemyDatabase` Auto-Find (missing from the database picker).

6.3 Tools cheat-sheet

Shortcut / Menu	Does
Ctrl+G	Open the Scene list popup.
Tools > Wave Editor	Author waves / segments / routes for a level.
Tools > Enemy Creator	Make a new enemy prefab variant.
Tools > Sheet Fetcher	Pull Upgrades / Characters / Towers from Sheets.
Tools > Localization	Bake UI text from a Sheet.

6.4 Key file index

File	Purpose
System/LevelInitializer.cs	Per-scene bootstrap; wires managers, loads the level.
System/ReferencesSO.cs + Refs.cs	Central registry (characters, colors, tower prefabs).
Managers/WaveManager.cs	Runtime spawning, routes, hordes, win condition.
Wave Related/WaveSO.cs	Wave data + WaveSpawnData + stat-mod modes.
Data/LevelData.cs	Routes, segments, wave groups (on the level prefab).
Enemies/Enemy.cs	Enemy movement, health, death, stat overrides.
Editor/Wave Editor/WaveEditorWindow.cs	The Wave Editor tool.
Editor/Enemy Creator/EnemyCreatorWindow.cs	The Enemy Creator tool.
Scripts/Editor/SheetFetcher.cs	The Sheet Fetcher tool.
Scripts/Editor/LocalizationSync.cs	The Localization baker.

End of document — SinSiege Developer Handover. Generated from the codebase with reportlab.